# Project: AI Tech Consultant

## Why this project

Software that cost a client $2,000 two years ago now sells for about $200, so a tech company must do 10 to 20 times more work to pay the same salaries. You will build the AI Tech Consultant that makes this possible.

Think of a small family business that needs a website change. Two years ago they hired a developer for $2,000. Today the owner can describe the change to Claude or Cursor and get most of it done and in next 6months its expected to be 100% finish-line-delivery.

Business owners will still hire developers, but they will not pay more than about $200, the highest plan of cursor / claude. That is a 90% drop in price.

Code is now the fast part. The slow part is everything around it: knowing the product inside out, answering every customer email quickly and correctly, and turning long client meetings into a clear scope of work. A good tech consultant does all of this today, by hand. No company can grow 10 to 20 times that.

Your job is to build an AI Tech Consultant: one system that knows the product, handles the inbox, and turns meetings into scopes of work. It must get better every week from the people who use it. You are not building it for one company. Build it so any software or service business could plug in its own help center, inbox and meetings.

## The project at a glance

The AI Tech Consultant is one project made of three connected tasks. Task 1 is the knowledge every other part depends on, so Tasks 2 and 3 must call Task 1 through its API, not copy its code.
1. The Knowledge Brain A memory and chatbot built from a real public help center, kept fresh every day. There will be an online list of web pages of the software company website or SAAS product knowledgebase, those pages can have images too, or a custom document or knowledge base making. Self evolving brain and soul.

2. The Inbox Agent An agent that reads customer emails, drafts replies and learns from
Python A public help center. TypeScript Task 1's API

3. Meeting to Scope of Work - every email a human Sends - A team of agents that turns client meeting transcripts into a scope of work, an estimate and a proposal uses Task 1's API

This project tests these skills. Use this list to prepare, not as a feature checklist.

| Skill | Task 1 | Task 2 | Task 3 |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Scraping, batching and scheduled jobs | Core | No | No |
| RAG, vector DB or knowledge graph | Core | Some | Some |
| Images as input (multimodal) | Core | No | No |
| Clean API design | Core | Core | Some |
| Tool calling and MCP | Some | Core | Core |
| Multi-agent and subagents | Some | Some | Core |
| Routing across models | Core | Core | Core |
| Evals for quality, cost and speed | Core | Core | Core |
| Structured output (JSON schemas) | Some | Core | Core |
| Self-improving prompts and agents | Some | Core | Some |
| Long context handling | Some | Some | Core |
| Human-in-the-loop and retries | Some | Core | Core |
| Simple, clear user screens | Core | Core | Core |

Task 1 — The Knowledge Brain

*Build the memory your users ask about anything: how the product works, how its settings behave, how refunds and cancellations are handled, and when the business goes out of its way to help a client. It must stay correct as the help center changes every day.*

**Language:** Python

Every software business has a help center, and most customer questions are already answered somewhere inside it. The trouble is that nobody can find the answer fast, and the help center keeps changing. For this task, everyone uses the same public help center, so we can test every student on the same questions: the Brilliant Directories Solution Center. It is a real product with hundreds of articles.

The help center has three levels. A **category** (also called a section) is a big area such as payments or design. Each category holds **folders**, and each folder is one topic. Each folder holds **articles**. Many articles include screenshots, and those images often carry the answer. The full list of pages lives in the sitemap:
support.brilliantdirectories.com/support/sitemap.xml.

What your Knowledge Brain must do:

- **Read everything once, politely.** The first full read must not overload the help center's server. Decide how you batch it, how fast you go, and what happens if it stops halfway.
- **Understand the images.** A screenshot is part of the answer. Your memory must know what each image shows and which article it belongs to.
- **Stay fresh every day.** Check the sitemap daily. Find pages that were added, updated or removed, and update only what changed. A removed article must stop appearing in answers.
- **Answer questions in a chatbot.** Give a correct answer, show which articles it came from, and say so honestly when it does not know.
- **Offer a clean API.** Tasks 2 and 3 will call your Knowledge Brain over its API. Another developer must be able to use it from your documentation alone.
- When the chatbot cannot answer, it creates a task for a human expert, and the expert's answer flows back into the memory.

## Task 2 — The Inbox Agent: replies that get better with every email sent

*Build an agent that reads every customer email, finds the facts from the Knowledge Brain, and drafts a reply a human can send in one click. Every time a human edits a draft or writes their own reply, the agent learns, until it can safely answer some kinds of email on its own.*

**Language:** TypeScript

Customers write in about everything: a how-to question, a new project request, a refund, a complaint, a quote, a follow-up on old work. A fast, correct reply wins the client, and a slow or wrong one loses them. Today a person reads each email, looks up the answer, writes the reply, and remembers who needs a follow-up.

Your Inbox Agent works like a smart assistant sitting beside that person. It understands what each email wants and gets the facts by calling the Knowledge Brain's API from Task 1. It then writes a draft in the company's voice. The human sends it, edits it, or writes their own. You decide what the agent learns from each of these, how it stores what it learned, and how you prove it is getting better and not worse.
Once humans have sent 20 drafts of the same kind in a row without changing them, that kind of email may go out on its own. You decide what "the same kind" and "without changing" mean. You also decide how the agent loses that freedom when quality drops.

There is no data pack. Write at least 50 realistic customer emails yourself, covering every kind of

request you can think of. We will also test your agent with emails you have never seen.

## Task 3 — Meeting to Scope of Work: one hour, not one week

*Turn messy client meeting transcripts into a scope of work, a developer estimate and a proposal the client can say yes to. When the next meeting changes things, update the scope instead of starting over.*

**Language:** Python

After a client meeting, someone writes the scope of work, developers estimate the hours, a senior checks it, and then the proposal goes out. Each step takes days. Clients speak loosely ("make the search better"), while developers need exact tasks. Items get missed, the scope grows quietly, and the company loses money on work it never quoted.

Design a team of AI agents for this. You choose the roles and decide what each one hands to the next. The team reads one or more transcripts and asks the questions a senior consultant would ask. It produces three things: a scope of work, a task list with hour estimates, and a client-ready proposal. Many requests can be solved with a setting the product already has, which costs the client far less than custom code. Your agents must check this by calling the Knowledge Brain from Task 1.

Then a second meeting changes some items. Show what changed, what is new, and what falls outside the agreed scope and should be quoted extra. Transcripts are long, so handle that well. Nothing reaches the client until a human approves it.

There is no data pack. Role-play at least 3 client meetings with a friend, record them, and use the transcripts. One of them must be a follow-up meeting that changes the first.

## Rules for students

Each student builds above capstone alone in 4 days and submits one repo, a short write-up and one demo video. These rules apply to everyone.

**Time:** 4 days from the moment you open this brief.

**Language:** use only the language named for each task. Python means modern Python: async, type hints, FastAPI or similar. TypeScript means strict mode, typed end to end, Node.js or Next.js. Do not mix languages inside one task.

IN CASE STUDENTS DON'T NOW TYPESCRIPT, THEY CAN USE PYTHON, BUT DON'T MIX.

STUDENTS KNOWING BOTH LANGUAGE, GETS 10% EXCESS SCORES IN FINAL SELECTION.

**Models:** Claude must be at the center. You may add other models where they help, but you must prove each choice with numbers. Even within Claude which model you will use, you need to justify your decision with 20-30 evals for each such decisions.

**AI tools:** Claude Code or Cursor or Antigravity or Codex, are allowed and expected. Use any one tool only and we will need to see your all chat conversations end to end, one or more chat threads.

*We judge your thinking, not your typing.*

**What to submit:**

| File | Length | What it must show |
|---|---|---|

| | | |
|---|---|---|
| GitHub repo | Any | One folder per task. The whole project runs with one command. Commits spread over the 4 days. |
| README.md | 1 page max | What it does and how to run it |

DESIGN.md 3 to 5 pages The problem in your own words, the features you chose and why, the features you dropped and why,

one diagram of how the three tasks connect, your model choices, what breaks and what you would build next

EVALS.md 2 to 3 pages At least 20 test cases per task, the models you compared, and quality, cost and speed for each

AI_LOG.md 1 page max Your 5 most useful prompts to Claude Code or Cursor, and one time you rejected the AI's answer and why

Demo video 10 minutes max Your face for the first 30 seconds, then all three tasks working together, including one case where it fails

**How to write:**

- Plain English, short sentences. Write for a busy founder, not a professor.
- Say why before what. Every feature needs a reason.
- Numbers, not adjectives. "92% correct at ₹0.40 per answer" beats "highly accurate and cheap". • Be honest about what does not work. Hiding a failure costs more marks than showing it. • Total writing is 6 to 10 pages. More pages do not earn more marks.

**Not allowed:** group work, copying another repo, or submitting work you cannot explain line by line in the live review.

## How you will be scored

The project is scored on 6 parts. The evaluator gives marks for each part directly, out of that part's maximum, using the last column as the guide for full marks. The six marks add up to a total out of 100.

| Part | Points | What we look for | What a full score looks like |
|---|---|---|---|
| 1. Problem thinking and feature ideas | 15 | Did you understand the business? Did you find needs nobody wrote down? | Features a real client would pay for, a clear list of what you dropped, and needs found without being told |
| 2. Design: UI/UX, architecture and agent design | 20 | Screens people can use, the right number of agents, clean handoffs between the three tasks | Simple screens a non-technical person understands, each agent has one job, and the diagram matches the code |
| 3. Code quality in the chosen language | 15 | Clean, typed, tested code in the language named for each task | Typed end to end, tests for the main paths, one-command run, sensible commits over 4 days |
| 4. Prompt design and self-improvement | 15 | Structured prompts, and a way for the system to get better | Prompts are versioned, outputs follow a schema, and human feedback improves the next run in a way you can measure and undo |
| 5. Evals for quality, cost, speed, reliability and safety | 20 | Did you measure, or just claim? What happens when things go wrong? | 20+ test cases per task, at least 2 models compared, cost per task shown, retries and human approval where it matters, and safe handling of bad input |
| 6. Writing and demo | 15 | Can a busy founder understand it in 10 minutes? | A short, honest write-up and a demo that shows a failure as well as a success |
| **Total** | **100** | | |

**The final result has two outcomes:**

| Total score | Result |
|---|---|
| 80 to 100, with every part at 60% or more | Fit |
| Below 80, or any part below 60% | Not a fit |

Two examples:

| Part | Max | Minimum (60%) | Student A | Student B |
|---|---|---|---|---|
| 1. Problem thinking | 15 | 9 | 13 | 11 |
| 2. Design | 20 | 12 | 18 | 14 |
| 3. Code quality | 15 | 9 | 12 | 13 |
| 4. Prompts and self-improvement | 15 | 9 | 12 | 11 |
| 5. Evals | 20 | 12 | 17 | 15 |
| 6. Writing and demo | 15 | 9 | 11 | 10 |
| **Total** | **100** | | **83: Fit** | **74: Not a fit** |

**Two more things decide the result:**

1. **A 30-minute live review.** You explain your code and make one small change live. If you cannot explain your own work, you do not pass, whatever your score. A co-event at BusinessLabs or NxtWave facility. 2. **Hidden test cases.** We run 10 to 30 test questions, emails and transcripts you have never seen. This catches systems built only to pass their own tests. Time followed by all submissions.